Advanced Data Analytics — Lecture 11

Simon Scheidegger
Department of Economics, University of Lausanne, Switzerland

November 24th, 2025 | 10:15 - 12:00: Internef 126 | 16:30 - 18:00: Anthropole 3185

Today's Roadmap

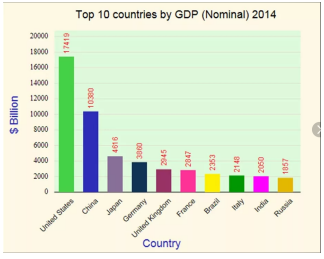
1. The curse of dimensionality and ways to deal with it
2. Bayesian Active Learning
3. Active subspaces
4. Principal Component Analysis

1. Motivation - the "curse of dimensionality"

- Modern dynamic economic models such (e.g. DSGE) are extremely rich to capture **all the effects of interest:**
- large stochastic shocks that lead to highly non-linear policies (e.g., rare disaster shocks).
- many agents that lead to a high-dimensional state space.
- ...
- Standard solution techniques such as log-linearization **often fail to deliver reliable results across the entire domain of interest.**
- The latter method may be useful to describe an economy that operates in normal times, but **fails** in the presence of nonlinearities such as occasionally binding constraints, among other types of salient features of the economic reality that the modelers would like to capture appropriately in their models.

Example — Heterogeneity in IRBC model

- Model trade imbalance
- FX rates
- ...



- How many regions does a minimal model have?
- Are policy functions smooth? (borrowing constraints)

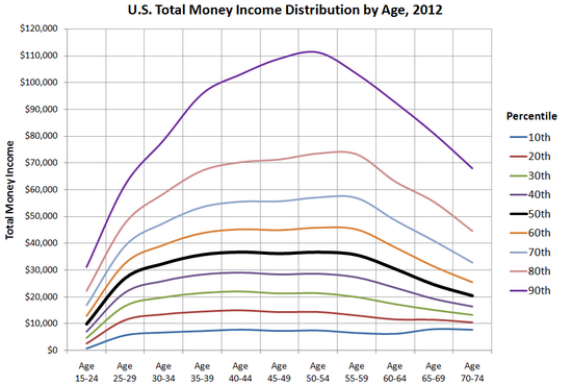
→ **Model heterogeneous & high-dimensional**

Example — Heterogeneity in Overlapping Generation models

To model e.g. social security:

- How many age groups?
- borrowing constraints?
- aggregate shocks?
- ...

→ **Model: heterogeneous & high-dimensional**



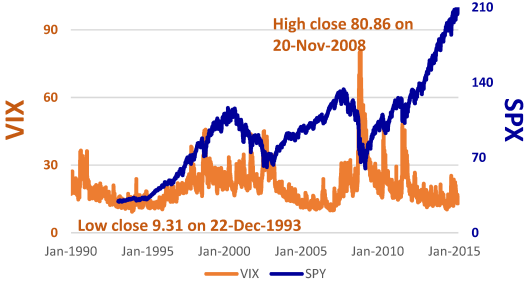
Financial markets: non-Gaussian returns

- Derivative contracts giving a right to buy or sell an underlying security.
  - **European** if exercise at expiration only.
  - **American** if exercise any time until expiration.

American options are extremely challenging:
$\Longrightarrow$ Dynamic optimization problem.

- Basic models do not describe dynamics accurately (e.g., Hull (2011)).
- Financial returns are often not Gaussian.
- Realistic models are hard to deal with, as they need many factors.

$\Longrightarrow$ Curse of dimensionality.



Dynamic Programming/Value Function Iteration

e.g. Stokey, Lucas & Prescott (1989), Judd (1998), …

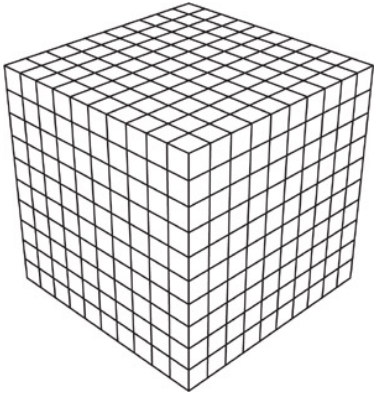
- Dynamic programming seeks a time-invariant policy function $\boldsymbol{p}$ mapping a state x_t into the control u_t such that for all $t \in \mathbb{N} \quad u_t = p(x_t)$ The solution is approached in the limit as $j \to \infty$ by iterations on:

$$V_{j+1}(x) = \max_u \{r(x,u) + \beta V_j(\tilde{x})\}$$

s.t.

$$\tilde{x} = g(x,u)$$

- x : grid point, describes your system. State-space potentially high-dimensional.
- 'old solution': high-dimensional function on which we interpolate.
- $\to \mathrm{N}^d$ points in ordinary discretization schemes.
- $\to$ Use-case for Gaussian Process regression.



How many dimensions is high dimensions?

How many dimensions is high dimensions?

Number of parameters (the dimension)	Number of model runs (at 10 points per dimension)	Time for parameter study (at 1 second per run)
1	10	10 sec
2	100	~ 1.6 min
3	1,000	~ 16 min
4	10,000	~ 2.7 hours
5	100,000	~ 1.1 days
6	1,000,000	~ 1.6 weeks
...	...	...
20	1e20	3 trillion years (240x age of the universe)

- Dimension reduction
  - Exploit symmetries, e.g., via the active subspace method
- Deal with #Points
  - Adaptive Sparse Grids
- High-performance computing
  - Reduces time to solution, but not the problem size

The Curse of Dimensionality

- Why is it hard (impossible) to learn functions in high dimensions?
- Methods relying on **local similarity measures** (e.g., kernels in the context of Gaussian Process Regression) have severe problems.
- "The curse of highly variable functions for local kernel machines", Bengio et al. (2006).

The reason is that the **Euclidean distance** is not a good "closeness" measure in high dimensions.

> *Most of the volume of a high-dimensional orange is in the skin, not in the pulp. If a constant number of examples is distributed uniformly in a highdimensional hypercube, beyond some dimensionality most examples are closer to a face of the hypercube than to their nearest neighbors.*

"A few useful things to know about machine learning", Pedro Domingos, (2012)

Volumes in high dimensions

- A lot of probability theory uses the idea of volumes.
- For example, the probability and expectation are usually defined as:

$$\mathbb{P}_\mu(A) = \int_A d\mu; \quad \mathbb{E}_\mu(f(X)) = \int_X f(x) d\mu(x)$$

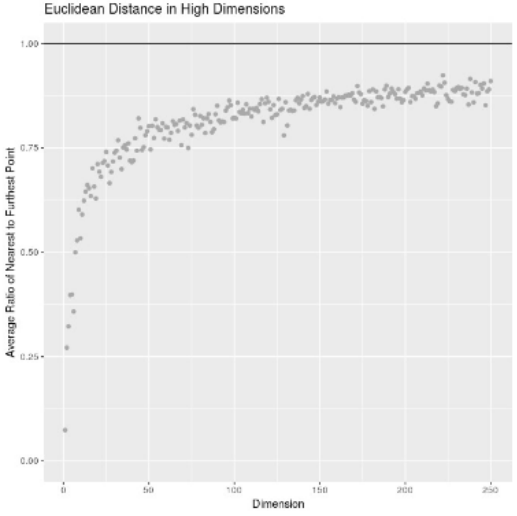
- where the differential element is usually a volume, $d_\mu(x) = p(x)dx_1 \dots dx_d$, and $p(x)$ is a density, but volumes are really weird in higher dimensions.
- The main idea is that our ideas of distance no longer make sense.

Volumes in high dimensions (II)

- Consider the following: Let's simulate a bunch of random normal points in many dimensions and plot the average ratio of the distance to nearest point to the distance to farthest point.

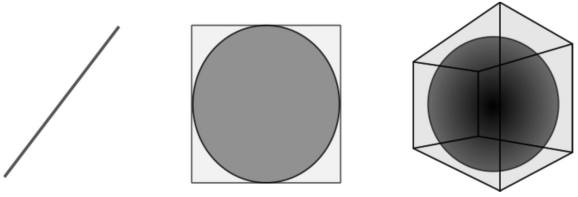
$$D = \sqrt{\sum_{i=1}^{d} (x_i)}$$

- This ratio tends to one, all points "look alike".
- → No good notion of "locality".
- → Problems, e.g., for GPR.

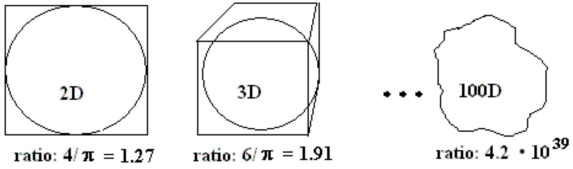


Volumes in high dimensions (III)

- On the same subject, consider a cube of unit lengths containing a sphere of unit radius in higher dimensions:

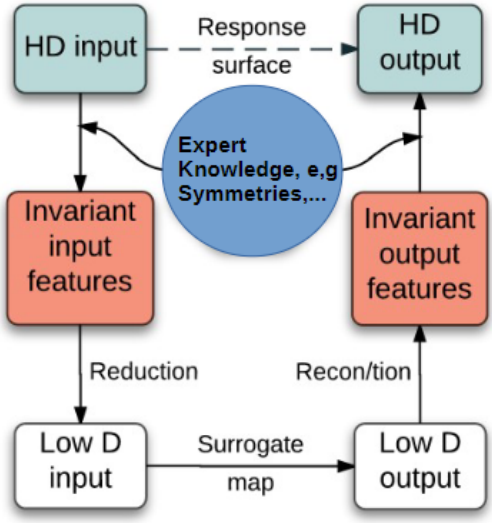


- The edges of the cube keep more and more volume away from the sphere (**cf. Sparse Grids - later in this course !!!**).



Dealing with high dimensions

- Generic term approach:
- Exploit invariances and symmetries induced by knowledge about the economics problem.
- Discover and exploit (non)-linear manifolds over which the response exhibits maximal variability.
- Here, we focus on:
  - HD input (already satisfying symmetries).
  - Discover and exploit linear manifold of maximal variability.



Abstract Objectives of Dimensionality Reduction

- data compression
- data visualization
- reduce the computational costs of the algorithms
- remove noise
- make the data-set easier to work with
- make the results easier to understand and interpret

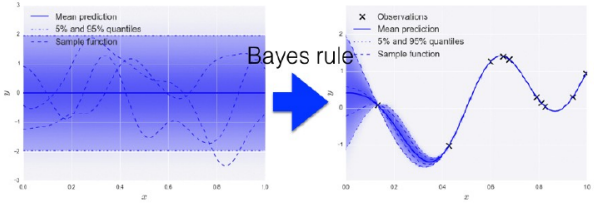
2. Bayesian Active Learning

- With the recent explosion of available data, you have can have millions of examples with a high cost to obtain labels.
- For instance, when trying to predict the sentiment of tweets, obtaining a training set can require immense manual labor
- But worry not, active learning comes to the rescue!
- In general, AL is a framework allowing you to increase classification performance by intelligently querying you to label the most informative instances.

Bayesian Active Learning

- With the recent explosion of available data, you have can have millions of examples with a high cost to obtain labels.
- For instance, when trying to predict the sentiment of tweets, obtaining a training set can require immense manual labour.
- But worry not, active learning comes to the rescue!
- In general, active learning is a framework allowing you to increase classification performance by intelligently querying you to label the most informative instances.
- BAL_2.ipynb and BAL.ipynb

Gaussian Processes in a Nutshell



Prior GP Posterior GP

Training set: $D = \{(\mathbf{x}_i, y_i) | i = 1, \ldots, n\}$

$$\begin{pmatrix} \mathbf{f} \\ \mathbf{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K} & \mathbf{K}_* \\ \mathbf{K}_*^T & \mathbf{K}_{**} \end{pmatrix} \right)$$

$$\begin{aligned} p(\mathbf{f}_* | \mathbf{X}_*, \mathbf{X}, \mathbf{f}) &= \mathcal{N}(\mathbf{f}_* | \boldsymbol{\mu}_*, \boldsymbol{\Sigma}_*) \\ \boldsymbol{\mu}_* &= \boldsymbol{\mu}(\mathbf{X}_*) + \mathbf{K}_*^T \mathbf{K}^{-1} (\mathbf{f} - \boldsymbol{\mu}(\mathbf{X})) \\ \boldsymbol{\Sigma}_* &= \mathbf{K}_{**} - \mathbf{K}_*^T \mathbf{K}^{-1} \mathbf{K}_* \end{aligned}$$

Test point = interpolation at X*

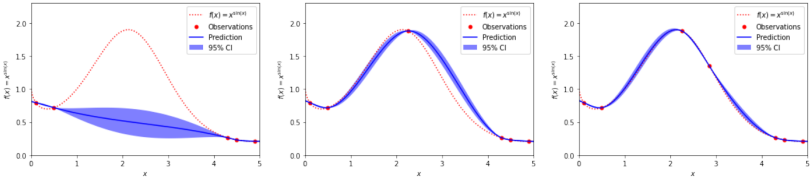
→ predictive mean $\boldsymbol{\mu}_* = \mathbb{E}(f_*)$

→ Confidence Intervals!

Where we have data, we have high confidence in our predictions.

Where we do not have data, we cannot be to confident about our predictions.

Bayesian Active Learning

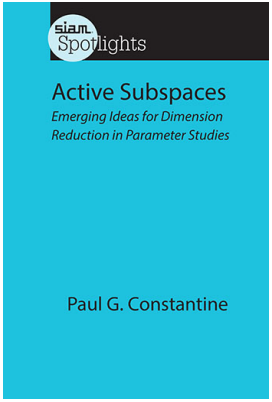


- Training of standard GPs scales as $\mathcal{O}(N^3)$ with the number of observations N.
- Runtime consequently would increase drastically with increasing N.
- Bayesian active learning: "add points where needed the most".

$$U(\tilde{x}) = \sigma_m \tilde{\mu}(\tilde{x}) + \frac{\sigma_v}{2} \log\left(\tilde{\sigma}(\tilde{x})\right) \tag{1}$$

$\sigma_m > 0$, $\sigma_v > 0$, and where $\tilde{\mu}$ and $\tilde{\sigma}$ are the predictive mean and variance of a GP, trained at input locations $\mathbf{X} = \{x_1, ..., x_n\}$, and evaluated at $\tilde{x}$, respectively.

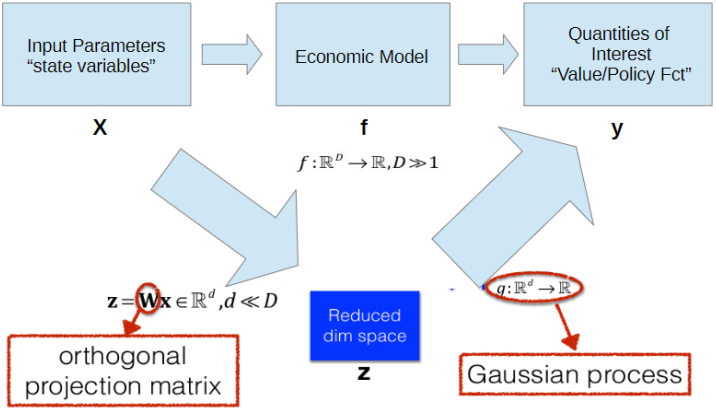
3. Active Subspaces



http://bookstore.siam.org/sl02/

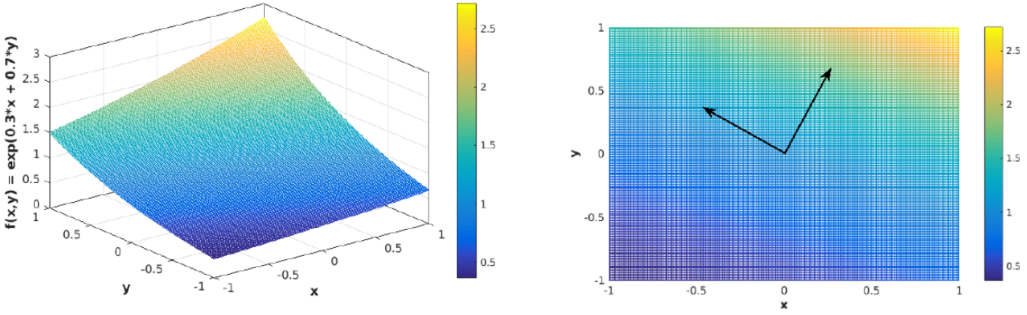
High Dimensions → Active Subspaces

(see, e.g., Constantine et al. (2013); Constantine (2015), with references therein)



(see, e.g., Constantine et. al (2013); Constantine (2015), with references therein)

Active Subspaces — intuitive example



Function varies most along [0.3, 0.7], and is constant in the orthogonal direction.

Discover active subspaces

(see, e.g., Constantine (2015), with references therein)

Step 1: Find **W**

$$\mathbf{C} = \mathbb{E}\left[\nabla_x f(\mathbf{x}) \nabla_x f(\mathbf{x})^T\right] \approx \frac{1}{N} \sum_{i=1}^{N} \nabla_x f\left(\mathbf{x}^{(i)}\right) \nabla_x f\left(\mathbf{x}^{(i)}\right)^T$$

Gradient info

"Mean-square directional derivative"

Note: there are also derivative-free ways of constructing **W**.

$$\mathbf{C} = \mathbf{W} \Lambda \mathbf{W}^T$$

Partition the eigendecomposition

- → Compute Eigenvalues, order them.
- → Look for "gaps.

$$\Lambda = \begin{bmatrix} \Lambda_1 & \\ & \Lambda_2 \end{bmatrix}, \qquad \mathbf{W} = \begin{bmatrix} \mathbf{W}_1 & \mathbf{W}_2 \end{bmatrix}, \qquad \mathbf{W}_1 \in \mathbb{R}^{m \times n}$$

Create a rotated coordinate system

$$\mathbf{x} = \mathbf{W}\mathbf{W}^T\mathbf{x} = \mathbf{W}_1\mathbf{W}_1^T\mathbf{x} + \mathbf{W}_2\mathbf{W}_2^T\mathbf{x} = \mathbf{W}_1\mathbf{q} + \mathbf{W}_2\mathbf{y}$$

GPs in active subspaces

Step 2: Regression $\longrightarrow$ $f(\mathbf{x}) \approx g(\boldsymbol{W}_1^T \mathbf{x})$.

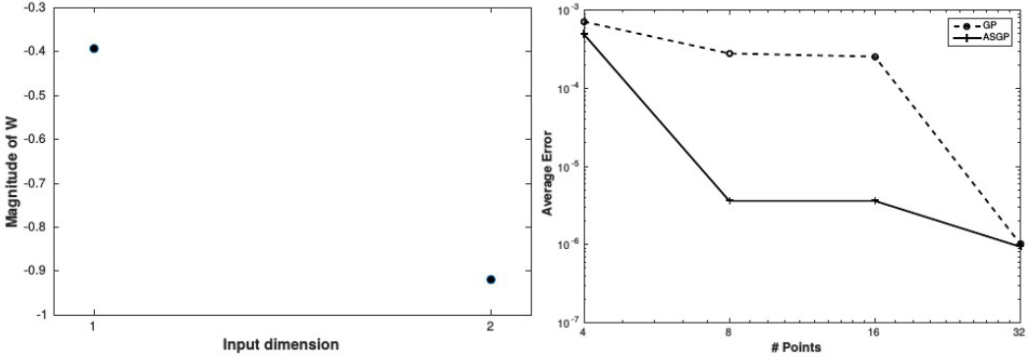
$$\mathcal{D}_{\text{projected}} = \left\{ \left(\mathbf{z}^{(i)} = \mathbf{W}\mathbf{x}^{(i)}, y^{(i)} = f(\mathbf{x}^{(i)}) \right) \right\}_{i=1}^{N}$$

$$g(\cdot) \sim \text{GP}\left(g(\cdot) \,|\, m(\cdot), k_0(\cdot,\cdot) \right).$$

$$g(\cdot) \,|\, \mathcal{D}_{\text{projected}} \sim \text{GP}\left(g(\cdot) \,|\, m^*(\cdot), k_0^*(\cdot,\cdot) \right)$$

Analytical example in 2D

`demo/AS_ex1.py`



$$f(x, y) = \exp(0.3x + 0.7y)$$

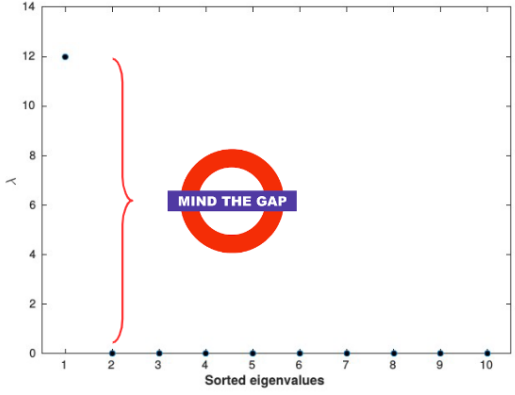
The left panel shows the projection matrix W of the 1-dimensional active subspace. **The right panel** displays a comparison of the interpolation error for 2-dimensional GPs and an 1-dimensional AS of varying resolution, respectively.

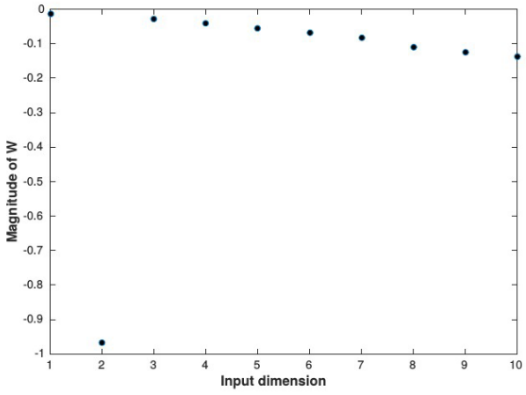
10d analytical example

demo/code/AS_ex2.py

$$f\colon [-1,1]^{10} \to \mathbb{R}$$

$$f(x_1,\ldots,x_{10}) = \exp(0.01x_1 + 0.7x_2 + 0.02x_3 + 0.03x_4 + 0.04x_5 + 0.05x_6 + 0.06x_7 + 0.08x_8 + 0.09x_9 + 0.1x_{10})$$



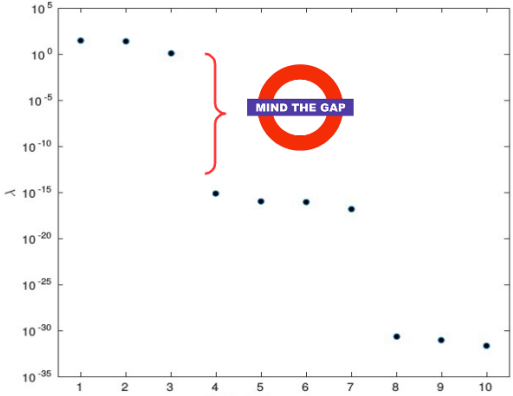


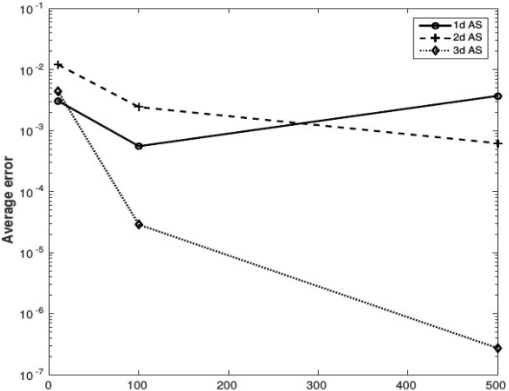
10d analytical example: Gap

demo/code/AS_ex3.py

$$f\colon [-1,1]^{10} \to \mathbb{R}$$
$$f(x_1,\dots,x_{10}) = \exp(0.01x_1 + 0.7x_2 + 0.02x_3 + 0.03x_4 + 0.04x_5 + 0.05x_6 + 0.06x_7 + 0.08x_8 + 0.09x_9 + 0.1x_{10})$$

ASGP surrogates of dimension $d = \{1, 2, 3\}$





Action Required

- Implement the function

$$f(x, y) = x * \sin(y)$$

- x from $[0, 1]$
- y from $[0, 1]$
- Use the Active subspace method.
- What are the Eigenvalues?
- How does the simplified approximation perform?

3. Principal Component Analysis

See: Machine Learning: An Algorithmic Perspective by Stephen Marsland

Data Compression:

- Reduce data from 2D to 1D

$$x^{(1)} \in R^2 \rightarrow z^{(1)}$$
$$x^{(2)} \in R^2 \rightarrow z^{(2)}$$
$$\vdots$$
$$x^{(m)} \in R^2 \rightarrow z^{(m)}$$

  e.g.: (work skill, work enjoyment) $\rightarrow$ work aptitude

- Reduce data from 3D to 2D

$$x^{(i)} \in R^3 \rightarrow z^{(i)} = \begin{bmatrix} z_1^{(i)} \\ z_2^{(i)} \end{bmatrix}$$

- Reduce data from 1000D to 100D.
- Reduce data from 1000D to 2D for data visualization.

Example: Going from 2d $\rightarrow$ 1d

Fig. from Marsland (2014)

x	y
2.00	-1.43
2.37	-2.80
1.00	-3.17
0.63	-1.80

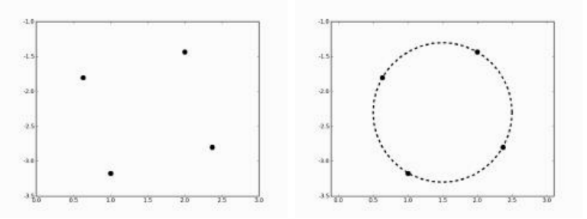


FIGURE 6.1 Three views of the same four points. *Left:* As numbers, where the links are unclear. *Centre:* As four plotted points. *Right:* As four points that lie on a circle.

PCA — an example

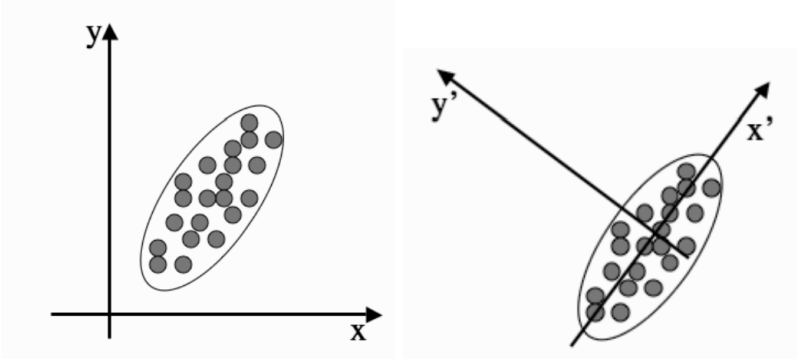


FIGURE 6.6 Two different sets of coordinate axes. The second consists of a rotation and translation of the first and was found using Principal Components Analysis.

→ **Idea: Remove dimensions with little variability**

Principal Component Analysis (PCA) problem formulation

- Reduce from 2-dimension to 1-dimension: Find a direction (a vector $u^{(1)} \in \mathrm{R}^2$) onto which to project the data so as to minimize the projection error.
- Reduce from n-dimension to k-dimension: Find k vectors $u^{(1)}, u^{(2)}, \cdots, u^{(k)} \in \mathbb{R}^n$ onto which to project the data so as to minimize the projection error.
- 3D $\rightarrow$ 2d, $K = 2$
- PCA is not linear regression $x \rightarrow y$

Regression:	vertical distance to y	a special variable y
PCA:	a shortest distance to projection surface	all variables $x_1, x_2, \cdots, x_n$ are treated symmetrically

Principal Component Analysis (PCA)

- By finding particular sets of coordinates axes $u^{(1)}, u^{(2)}, \cdots, u^{(k)} \in \mathbb{R}^n$, it will became clear that some of the dimensions are not required.
- In fact, it can make the results better, since we are often removing some of the noise in the data.
- The question is how to choose the directions $u^{(1)}, u^{(2)}, \cdots, u^{(k)} \in \mathbb{R}^n$
- The idea of the principal component is that it is a direction in the data with the largest variation.

Data preprocessing

- Training set: $x^{(1)}, x^{(2)}, \cdots, x^{(k)}$
- Preprocessing (feature scaling/mean normalization):

$$\mu_j = \frac{1}{m} \sum_{i=1}^{m} x_j^{(i)}$$

- Replace each $x_j^{(i)}$ with $x_j^{(i)} - \mu_j$.
- If different features on different scales (e.g., x_1 = size of house, x_2 = number of bedrooms), scale features to have comparable range of values:

$$x_j^{(i)} \text{ with } x_j^{(i)} - \mu_j.$$

s_j is some measure of deviation like range $= \max - \min$ or standard deviation.

The PCA Algorithm

- Reduce data from n dimensions to k dimensions
- Write m data points $x^{(i)} = \left(x_1^{(i)}, x_2^{(i)}, \ldots, x_n^{(i)}\right)$ as row vectors.
- Matrix X will have size $m \times n$.
- Center the data by subtracting off the mean of each column.

The PCA Algorithm

- Compute **covariance matrix**:

$$C = \frac{1}{m} \sum_{i=1}^{n} \left(x^{(i)}\right)^T \left(x^{(i)}\right)$$

$\left(x^{(i)}\right)^T$ is a $n \times 1$ vector, $x^{(i)}$ is a $1 \times n$ vector, then C will be a $n \times n$ matrix.

The PCA Algorithm

- Rotate the data $Y = P^T X$.
- P^T is a rotation matrix and P is choose so that the covariance matrix of Y is diagonal:

$$\text{cov}(Y) = \text{cov}\left(P^T X\right) = \begin{pmatrix} \lambda_1 & & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & \cdots & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & & 0 & \cdots & \lambda_N \end{pmatrix}$$

- Thus $\lambda P = CP$

The PCA Algorithm

- Compute the eigenvalues and eigenvectors of C, so $V^{-1}CV = D$, where V holds the eigenvectors of C and D is the $n \times n$ diagonal eigenvalue matrix:

$$\text{evecs} = U = \left[\; u^{(1)}; \quad u^{(2)}; \quad \cdots ; \quad u^{(n)} \; \right]$$

- Sort the columns of D into order of decreasing eigenvalues, and apply the same order to the columns of V.
- **Leave k dimensions in the data:**

$$U_{\text{reduce}} = \left[\; u^{(1)}; \quad u^{(2)}; \quad \cdots ; \quad u^{(k)} \; \right]$$

The PCA Algorithm

- The new components $z^(i)$ will be calculated as:

$\left(z^{(i)}\right)^T = \left(U_{\text{reduce}}\right)^T \left(x^{(i)}\right)^T$ or $\quad z^{(i)} = x^{(i)} U_{\text{reduce}}$

U_{reduce} is a $n \times k$ matrix, $x^{(i)}$ is a $1 \times n$ vector, then $z^{(i)}$ will be a $1 \times k$ vector.

$\rightarrow$ demo/Principal_ComponentAnalysis_01.ipynb

Alternative derivation of PCA: Minimum error

We introduce a complete orthonormal set of D-dimensional basis vectors $\{u_i\}$ where $i = 1, \ldots, D$ that satisfy

$$\mathbf{u}_i^{\mathrm{T}}\mathbf{u}_j = \delta_{ij}. \quad \alpha_{nj} = \mathbf{x}_n^{\mathrm{T}}\mathbf{u}_j \quad \mathbf{x}_n = \sum_{i=1}^{D} \alpha_{ni}\mathbf{u}_i$$

$$\mathbf{x}_n = \sum_{i=1}^{D} \left(\mathbf{x}_n^{\mathrm{T}}\mathbf{u}_i\right)\mathbf{u}_i$$

$$\widetilde{\mathbf{x}}_n = \sum_{i=1}^{M} z_{ni}\mathbf{u}_i + \sum_{i=M+1}^{D} b_i\mathbf{u}_i$$

Alternative derivation of PCA: Minimum error

Minimize

$$J = \frac{1}{N}\sum_{n=1}^{N} \|\mathbf{x}_n - \widetilde{\mathbf{x}}_n\|^2 .$$

$$z_{nj} = \mathbf{x}_n^{\mathrm{T}}\mathbf{u}_j \quad \text{where } j = 1, \ldots, M.$$

setting the derivative of J with respect to b_i to zero

$$\longrightarrow \quad b_j = \overline{\mathbf{x}}^{\mathrm{T}}\mathbf{u}_j \quad \text{where } j = M+1, \ldots, D$$

$$\mathbf{x}_n - \widetilde{\mathbf{x}}_n = \sum_{i=M+1}^{D} \left\{ \left(\mathbf{x}_n - \overline{\mathbf{x}}\right)^{\mathrm{T}} \mathbf{u}_i \right\} \mathbf{u}_i$$

Alternative derivation of PCA: Minimum error

$$J = \frac{1}{N}\sum_{n=1}^{N}\sum_{i=M+1}^{D}\left(\mathbf{x}_n^{\mathrm{T}}\mathbf{u}_i - \overline{\mathbf{x}}^{\mathrm{T}}\mathbf{u}_i\right)^2 = \sum_{i=M+1}^{D}\mathbf{u}_i^{\mathrm{T}}\mathbf{S}\mathbf{u}_i.$$

constrained minimization otherwise we will obtain the vacuous result $\mathbf{u}_i = 0$. minimize $J = \mathbf{u}_2^{\mathrm{T}}\mathbf{S}\mathbf{u}_2$, subject to the normalization constraint $\mathbf{u}_2^{\mathrm{T}}\mathbf{u}_2 = 1$

$$\widetilde{J} = \mathbf{u}_2^{\mathrm{T}}\mathbf{S}\mathbf{u}_2 + \lambda_2\left(1 - \mathbf{u}_2^{\mathrm{T}}\mathbf{u}_2\right)$$

Setting the derivative with respect to $\mathbf{u}_2$ to zero, we obtain $\mathbf{S}\mathbf{u_2} = \lambda_2\mathbf{u}_2$ $\mathbf{u}_2$ is an eigenvector of $\mathbf{S}$ with eigenvalue λ_2.

Alternative derivation of PCA: Minimum error

The general solution to the minimization of J for arbitrary D and arbitrary $M < D$ is obtained by choosing the $\{\mathbf{u}_i\}$ to be eigenvectors of the covariance matrix given by

$$\mathbf{S}\mathbf{u}_i = \lambda_i \mathbf{u}_i$$

where $i = 1, \ldots, D$, and as usual the eigenvectors $\{\mathbf{u}_i\}$ are chosen to be orthonormal. The corresponding value of the distortion measure is then given by

$$J = \sum_{i=M+1}^{D} \lambda_i$$

Alternative derivation of PCA: Minimum error

- This is simply the sum of the eigenvalues of those eigenvectors that are orthogonal to the principal subspace.
- We therefore obtain the minimum value of J by selecting these eigenvectors to be those having the D - M smallest eigenvalues, and hence the eigenvectors defining the principal subspace are those corresponding to the **M** largest eigenvalues.
- Although we have considered $M < D$, the PCA analysis still holds if $M = D$, in which case there is no dimensionality reduction but simply a rotation of the coordinate axes to align with principal components.

Reconstruction from compressed representation

$\left(z^{(i)}\right)^T = \left(U_{\text{reduce}}\right)^T \left(x^{(i)}\right)^T$ or $z^{(i)} = x^{(i)} U_{\text{reduce}}$ U_{reduce} is a $n \times k$ matrix, $x^{(i)}$ is a $1 \times n$ vector, then $z^{(i)}$ will be a $1 \times k$ vector.

Choosing k (number of principal components)

- Average squared projection error: $\frac{1}{m}\sum_{i=1}^{n}\left\|x^{(i)}-x_{\text{approx}}^{(i)}\right\|^2$
- Total variation in the data: $\frac{1}{m}\sum_{i=1}^{n}\left\|x^{(i)}\right\|^2$
- Typically, choose k to be smallest value so that

$$\frac{\frac{1}{m}\sum_{i=1}^{n}\left\|x^{(i)}-x_{\text{approx}}^{(i)}\right\|^2}{\frac{1}{m}\sum_{i=1}^{n}\left\|x^{(i)}\right\|^2}\leq \eta = 0.01 \quad (1\%)$$

- 99% of variance is retained
- If $\eta = 0.05(5\%)$, then 95% of variance is retained.
- If $\eta = 0.1(10\%)$, then 90% of variance is retained.

Choosing k (number of principal components)

1. Try PCA with $k = 1$
2. Compute $U_{\text{reduce}}, z^{(1)}, z^{(2)}, \cdots, z^{(m)}, x_{\text{approx}}^{(1)}, \cdots, x_{\text{approx}}^{(m)}$
3. Check if
$$\frac{\frac{1}{m}\sum_{i=1}^{n}\left\|x^{(i)} - x_{\text{approx}}^{(i)}\right\|^2}{\frac{1}{m}\sum_{i=1}^{n}\left\|x^{(i)}\right\|^2} \leq 0.01?$$
4. If not, then try PCA with $k = 2$, with $k = 3$ and so on until, for example, for $k = 17$ check 3 will be satisfied

Choosing k (number of principal components)

- Another algorithm to choose k is to use a matrix

$$D = \begin{pmatrix} \lambda_1 & & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & \cdots & 0 \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ 0 & & 0 & \cdots & \lambda_N \end{pmatrix}$$

- For given k the retained variance is

$$1 - \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i}$$

- Pick smallest value of k for which

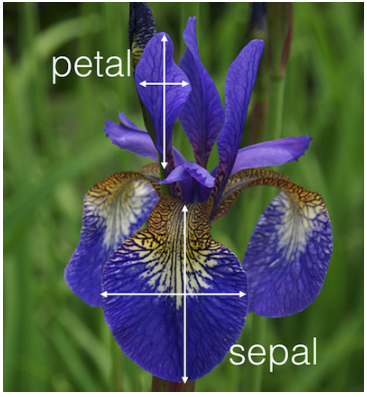
$$\frac{\sum_{i=1}^{k} \lambda_i}{\sum_{i=1}^{n} \lambda_i} \geq 0.99$$

(99% of variance retained)

Example for Principal Component Analysis (PCA): Iris data

demo/iris/iris_pca.ipynb

- Downloaded the file from here: https://archive.ics.uci.edu/ml/datasets/iris
- https://en.wikipedia.org/wiki/Iris_flower_data_set
-
- We have 150 iris flowers.
- For each flower we have 4 measurements
- sepal length, sepal width, petal length, petal width giving 150 points $x^{(1)}, \ldots, x^{(150)} \in \mathbb{R}^4$
- The flowers belong to three different species: 0 : setosa, 1: versicolor, 2: virginica



Kernel PCA (read at home)

- One problem with PCA is that it assumes that the directions of variation are all straight lines.
- This is often not true.
- The Kernel PCA uses the kernel trick to get around this problem.
- We apply a (possible non-linear) function $\Phi(\bullet)$ to each data point x that transform the data into the kernel space, and then perform normal linear PCA in that space.

Kernel PCA (read at home)

- The covariance matrix is defined in the kernel space:

$$C = \frac{1}{N}\sum_{n=1}^{N} \Phi\left(x_n\right) \Phi\left(x_n\right)^T,$$

- which produces the eigenvector equation:

$$\lambda\left(\Phi\left(x_i\right) P\right) = \left(\Phi\left(x_i\right) CP\right) \quad i = 1 \cdots N,$$

- Where

$$P = \sum_{j=1}^{N} \alpha_j \Phi\left(x_j\right)$$

- are the eigenvectors of the original problem and the α_j will turn out to be the eigenvectors of the kernelized problem. The projection of a new point x into the kernel PCA space:

$$\left(P_k \Phi(x)\right) = \sum_{i=1}^{N} \alpha_i^k \left(\Phi\left(x_i\right) \Phi\left(x_j\right)\right.$$

Kernel PCA (read at home)

demo/plot_kernel_pca.ipynb, demo/Kernel_PCA_example.ipynb

- We produce an $N \times N$ kernel matrix $\mathrm{K}_{i,j} = (\Phi(x_i)\,\Phi(x_j))$ and get the equation $N\lambda\alpha = K\alpha$.
- There are three different types of basic function and the kernel that corresponds to them:
- linear $K(x, y) = xy$
- radial basis function expansions of the x_k with parameter σ and kernel: $K(x, y) = \exp\left(-(x-y)^2/2\sigma^2\right)$
- polynomials up to some degree s in the elements x_k of the input vectors with kernel

$$K(x, y) = (a + xy)^s$$

Active Subspaces versus PCA

- Some comment on the similarities and didiscrepancies between PCA and AS:
- PCA identifies a projection of the input space.
- The goal of this projection, however, is not the same as in AS.
- PCA picks the projection that minimizes the mean square reconstruction error of the input x that is, it minimizes $E\left[\left\|x - \mathbf{W}\left(\mathbf{W}^\top x\right)\right\|\right]$, where the expectation is with respect to the input-generating distribution.
- AS has an objective that is very different to that of PCA:
- AS focuses on finding a W that allows us to approximate $f(x)$ with a function of the form $\mathbf{h}(\mathbf{Wx})$ as well as possible.
- Even though AS has not (yet) been formulated to directly minimize the mean square error $\mathrm{E}\left[(\mathrm{f}(x) - \mathrm{h}(\mathrm{Wx}))^2\right]$, it was shown by Constantine that the mean square error is bounded by a term proportional to the sum of the neglected eigenvalues of C.

$\rightarrow$ PCA focuses on finding the best linear projection that allows the reconstruction of the input, whereas AS focuses on the search for the best linear projection that enables the reconstruction of the response surface $f(x)$.

Questions?